

The Alberta Buck - Formal Correctness Proofs (v1.0)

Rerandomization, Re-Encryption, and Notes Spend Predicates

Perry Kundert

2026-04-22



The Alberta Buck identity system (Identity, Examples) rests on two cryptographic transformations: the *Identity Fountain* rerandomizes a Pointcheval-Sanders signature to produce an unlinkable credential, and the **approve** handshake re-encrypts the credential's ElGamal ciphertext under the counterparty's public key with a Chaum-Pedersen proof of equality. Everything downstream – bilateral transfers, the BUCK Notes spend circuits (Notes), Identity Guardian authorizations, regulatory recovery – inherits its correctness from these two primitives.

This document states and proves the correctness properties that the rest of the series relies on:

- **Theorem 1:** rerandomization preserves PS signature validity (completeness).
- **Theorem 2:** a uniform random rerandomization scalar yields a signature statistically identical to a fresh one, independent of the original (unlinkability).
- **Theorem 3:** an honest Chaum-Pedersen prover always verifies (completeness).
- **Theorem 4:** from two accepting transcripts with distinct challenges an efficient extractor recovers the witness (special soundness, 2-extractability).
- **Theorem 5:** the sigma-protocol transcript is perfectly simulatable given only public inputs and the challenge (HVZK).
- **Theorem 6:** combined, an accepted on-chain Chaum-Pedersen proof implies that the re-encrypted ciphertext carries the same issuer-certified identity point $M = m \cdot G$ as the registered credential (end-to-end re-encryption correctness).
- **Theorem 7:** at mint the issuer addresses the recipient *Identity* point M_{rec} (not an account); the ciphertext decrypts under m_{rec} to the intended point, a mint-binding forces it over the issuer's *own* registered Identity (anti-framing), and the issuer never learns m_{rec} .

- **Theorem 8** (deposit-coupling soundness, A1/A2): an accepted addressed spend proves the depositing account is bound to the registered Identity scalar m_{rec} AND the note ciphertext decrypts under it to a registered member – account-agnostic (recovery, not finality, on key loss) and collusion-closed.
- **Theorem 9** (depositor-binding soundness, B1): an accepted bearer spend proves the depositor’s Identity M_{dep} is registered AND encrypted for the public issuer, who alone names it; a bogus M_{dep} is a non-member and reverts.
- **Theorem 10**: the flavor-agnostic nullifier $nf = \text{Poseidon}_3(\rho, idHash, tag)$ deterministically prevents double-spend and, as a PRF over the secret opening, hides the $cm \leftrightarrow nf$ link from any party without the opening (transaction-graph privacy against bulk harvest).
- **Theorem 11**: every accepted spend is mutually decryptable – both counterparties name the other as a *registered* Identity, *enforced* by the membership half (an un-nameable counterparty is un-spendable) – while no third party names either or links mint to spend.
- **Theorem 12** (note-binding re-encryption tie, A1/A2): the ciphertext a depositor supplies at spend is provably a re-encryption – under the authenticated recipient Identity – of the ciphertext the *spent note itself* committed at mint. This pins Theorems 8 and 11 to the specific note: a stolen opening cannot be redeemed with a self-addressed substitute, and an un-nameable A2 note stays un-spendable even against active ciphertext substitution.

Every theorem is accompanied by an executable `py_ecc` sanity check on BN254. The proofs assume only the q-SDH assumption (for PS signature unforgeability), the discrete logarithm assumption on $\text{alt_bn128 } G_1$, the random oracle model (for the Fiat-Shamir transform and Poseidon hashing), and knowledge soundness of the underlying SNARK proof system (for Theorems 8-9 and 12) – all standard assumptions that Ethereum’s security or any zk-SNARK rollout already requires. (PDF, Text)

Contents

1	Preliminaries	5
1.1	Notation	5
1.2	Hardness Assumptions	5
2	Part I: Pointcheval-Sanders Signature Rerandomization	6
2.1	The Construction	6
2.2	Theorem 1 (Completeness of Rerandomization)	6
2.3	Theorem 2 (Statistical Unlinkability)	6
2.4	Sanity Check (py_ecc on BN254)	7
3	Part II: Chaum-Pedersen Re-Encryption Correctness	9
3.1	Setup	9
3.2	The Statement	9
3.3	Protocol (Fiat-Shamir)	9
3.4	Theorem 3 (Completeness)	10
3.5	Theorem 4 (Special Soundness / 2-Extractability)	10
3.6	Theorem 5 (Honest-Verifier Zero-Knowledge)	11
3.7	Sanity Check (py_ecc on BN254)	11
4	Part III: End-to-End Re-Encryption Correctness	14
4.1	Chain of Trust	14
4.2	Theorem 6 (End-to-End Re-Encryption Correctness)	14
4.3	Why This Suffices for Every Downstream Use Case	14
4.4	The EOA Transfer Mirror (transfer / transferFrom)	15
4.5	Sanity Check (py_ecc on BN254)	15
5	Part IV: BUCK Notes Spend Predicates (A1, A2, B1)	17
5.1	Setup	17
5.2	Theorem 7 (Mint Addressing and Anti-Framing)	17
5.3	Theorem 8 (Deposit-Coupling Soundness – A1, A2)	18
5.4	Theorem 9 (Depositor-Binding Soundness – B1)	19
5.5	Theorem 10 (Nullifier and Double-Spend Prevention)	19
5.6	Theorem 11 (Mutual Decryptability and Transaction-Graph Privacy)	20
5.7	Theorem 12 (Note-Binding Re-Encryption Tie – A1, A2)	20
5.8	Sanity Check (py_ecc on BN254)	22
5.9	Worked Example: An Identity-M Spend End to End	23
5.10	Mint-Circuit Status: Phase 7-bis Closure of the Earlier Open Questions	23
5.10.1	Closure 1 (Field-Overflow / Value-Conservation Range Check) – <i>Discharged in circuit</i>	23
5.10.2	Closure 2 (Single transferFrom $\leftrightarrow$ N Commitments) – <i>Discharged by SNARK binding</i>	24
5.10.3	Closure 3 (Spend-Circuit Trio) – <i>Mint half discharged; nullifier + spend-side value conservation deferred to spend.circom</i>	24
6	Security Assumptions Summary	25

7	Implementation Notes	26
7.1	What the Contract Must Enforce	26
7.2	What the Prover Must Enforce	26
7.3	Scope Not Covered Here	26
7.4	The Long-Horizon Threat: Harvest-Now, Decrypt-Later	27

DRAFT

1 Preliminaries

1.1 Notation

Symbol	Meaning
q	Prime order of G_1, G_2, G_T on BN254
G, g_2	Generators of G_1 and G_2
$e : G_1 \times G_2 \rightarrow G_T$	Bilinear, non-degenerate pairing (type-3)
O	Point at infinity (group identity)
$\mathbb{Z}_q^*$	Non-zero scalars mod q
$(x, y)/(X, Y)$	Issuer secret / public key: $X = xg_2, Y = yg_2$
$m \in \mathbb{Z}_q$	Identity scalar, $m = H(\text{identity_data}) \bmod q$
$M = m \cdot G \in G_1$	Identity point (the value every ElGamal ciphertext encrypts)
$\sigma = (\sigma_1, \sigma_2) \in G_1^2$	PS signature: $\sigma_1 = h, \sigma_2 = (x + my)h$
(sk, pk)	Identity key pair on G_1 : $pk = sk \cdot G$
$E = (R, C) \in G_1^2$	ElGamal ciphertext: $R = rG, C = M + r \cdot pk$
$H(\cdot)$	Cryptographic hash modelled as a random oracle (Fiat-Shamir)

All group operations in this document are written additively. BN254 has cofactor 1 on G_1 , so every non-identity $P \in G_1$ has order q ; there are no small-subgroup elements.

1.2 Hardness Assumptions

Four standard assumptions underpin the theorems in this document. None is specific to Alberta Buck – each one is already required by Ethereum’s own security model or by any pairing-based / zk-SNARK construction built on the `ecPairing` precompile.

1. **Discrete Logarithm (DLog) on G_1** : given $P, Q \in G_1$, finding $\alpha \in \mathbb{Z}_q$ with $Q = \alpha P$ is infeasible. Protects ElGamal IND-CPA security, Schnorr unforgeability, and Chaum-Pedersen soundness.
2. **q-Strong Diffie-Hellman (q-SDH)**: the PS signature scheme is existentially unforgeable under chosen-message attack (EUF-CMA) assuming q-SDH¹. We use this only as a **stated** assumption about the PS primitive; none of our proofs below reduce to it directly.
3. **Random Oracle Model (ROM)**: the Fiat-Shamir transform is applied to sigma protocols whose security is proved in the ROM, with the hash function H modelled as a random oracle bound to all public inputs and context (caller address, chain id, etc.). Poseidon (commitment hash and nullifier PRF) is also modelled as a random oracle for the BUCK Notes proofs in Part IV.
4. **Knowledge soundness of the underlying zk-SNARK proof system** (Part IV only): the SNARK system used for BUCK Notes spend circuits (Groth16, PLONK, or any knowledge-sound construction over BN254) admits a polynomial-time witness extractor on every accepting proof. This is a system-specific assumption depending on the chosen scheme’s trusted setup or universal setup.

The rerandomization proofs (Part I) are *unconditional*: Theorem 1 follows from bilinearity alone, and Theorem 2 is information-theoretic. The Chaum-Pedersen proofs (Part II) are interactive-sigma-protocol facts; the NIZK corollary (Part II, end) invokes the ROM. Part IV (Notes spend predicates) additionally invokes assumption (4) – knowledge soundness of the underlying SNARK system – to lift the sigma-protocol guarantees of Theorems 4-5 into the SNARK setting.

¹Pointcheval, D. and Sanders, O. "Short Randomizable Signatures." CT-RSA 2016, LNCS 9610, pp. 111-126. Theorem 4.2 establishes EUF-CMA security under the q-SDH assumption in bilinear groups.

2 Part I: Pointcheval-Sanders Signature Rerandomization

2.1 The Construction

A Pointcheval-Sanders signature on $m \in \mathbb{Z}_q$ is a pair $\sigma = (\sigma_1, \sigma_2) \in G_1^2$ with $\sigma_1 \neq O$ satisfying the pairing equation

$$e(\sigma_1, X + m \cdot Y) = e(\sigma_2, g_2).$$

The issuer produces σ by choosing a fresh $h \in G_1 \setminus \{O\}$ and setting $\sigma_1 = h$, $\sigma_2 = (x + my)h$. *Rerandomization* is the operation

$$\sigma' = (t \cdot \sigma_1, t \cdot \sigma_2) \quad t \in \mathbb{Z}_q^*.$$

2.2 Theorem 1 (Completeness of Rerandomization)

Statement. For every valid PS signature σ on m under issuer key (X, Y) , and for every $t \in \mathbb{Z}_q^*$, the pair $\sigma' = (t\sigma_1, t\sigma_2)$ is also a valid PS signature on m under the same issuer key.

Proof. By bilinearity of e :

$$\begin{aligned} e(\sigma'_1, X + mY) &= e(t\sigma_1, X + mY) && \text{(definition)} \\ &= e(\sigma_1, X + mY)^t && \text{(bilinearity)} \\ &= e(\sigma_2, g_2)^t && \text{(original validity)} \\ &= e(t\sigma_2, g_2) && \text{(bilinearity)} \\ &= e(\sigma'_2, g_2). \end{aligned}$$

Non-degeneracy: G_1 has prime order q , so $t\sigma_1 = O$ iff $t \equiv 0 \pmod{q}$ or $\sigma_1 = O$. Both are excluded by hypothesis ($t \in \mathbb{Z}_q^*$ and $\sigma_1 \neq O$ by PS validity). Therefore $\sigma'_1 \neq O$ and the pair satisfies the full PS validity predicate. ■

Implementation guard. The registration contract MUST enforce $\sigma'_1 \neq O$. Without this check, a prover can submit $\sigma' = (O, O)$ (obtainable by picking $t = 0$); the pairing equation then reduces to $1 = 1$ in G_T for every m , allowing a fabricated credential to pass registration. Theorem 1's completeness argument explicitly depends on $t \neq 0$; the contract's guard is the runtime witness of this hypothesis.

2.3 Theorem 2 (Statistical Unlinkability)

Statement. Let σ be any fixed valid PS signature on m . If t is sampled uniformly from $\mathbb{Z}_q^*$, then the distribution of $\sigma' = (t\sigma_1, t\sigma_2)$ is identical to the distribution of $\text{PS.Sign}(sk_{\text{issuer}}, m)$ for h sampled uniformly from $G_1 \setminus \{O\}$, and is *independent* of σ .

Proof. Since G_1 is cyclic of prime order q and $\sigma_1 \neq O$, the map

$$\varphi_\sigma : \mathbb{Z}_q^* \rightarrow G_1 \setminus \{O\}, \quad t \mapsto t\sigma_1$$

is a bijection (it is the restriction to non-zero scalars of the group isomorphism $\mathbb{Z}_q \rightarrow \langle \sigma_1 \rangle = G_1$). Hence σ'_1 is uniform on $G_1 \setminus \{O\}$ regardless of the specific σ_1 we started with.

Given $\sigma'_1 = h'$, the second component is forced: $\sigma'_2 = t\sigma_2 = t(x + my)\sigma_1 = (x + my)(t\sigma_1) = (x + my)h'$. That is exactly what a fresh PS.Sign with random h' would output.

So the joint distribution of σ' matches the fresh-signature distribution, and the map $\sigma \mapsto \sigma'$ destroys all information about σ beyond the fact that both sign the same m . This is *statistical* (information-theoretic) unlinkability, not merely computational. ■

Remark. The argument uses only that G_1 has prime order; no cryptographic assumption is invoked. A cofactor > 1 would leak subgroup membership of σ_1 ; the prime-order property of BN254 G_1 is therefore load-bearing.

2.4 Sanity Check (py_ecc on BN254)

```
import secrets, hashlib
from py_ecc.bn128 import bn128_curve as bc
from py_ecc.bn128 import pairing

ORDER = bc.curve_order
G1, G2 = bc.G1, bc.G2
multiply, add, neg, eq = bc.multiply, bc.add, bc.neg, bc.eq

def rand():
    return secrets.randbelow(ORDER - 1) + 1

# Issuer key pair
x, y = rand(), rand()
X, Y = multiply(G2, x), multiply(G2, y)

# Alice's identity and PS signature sigma = (h, (x + m y) h)
m = int(hashlib.sha256(b"Alice Johnson | Alberta | AIC-2026").hexdigest(), 16) % ORDER
h = multiply(G1, rand())
sigma = (h, multiply(h, (x + m * y) % ORDER))

# Rerandomize with t != 0
t = rand()
sigma_p = (multiply(sigma[0], t), multiply(sigma[1], t))

# --- Theorem 1: completeness ---
lhs = pairing(add(X, multiply(Y, m)), sigma_p[0])
rhs = pairing(G2, sigma_p[1])
assert lhs == rhs, "Theorem 1 FAILED: rerandomized signature does not verify"
assert not eq(sigma_p[0], bc.Z1), "Theorem 1 guard: sigma'_1 == 0"
print("Theorem 1 (completeness):          PASS")

# --- Theorem 2: fresh rerandomization looks like a fresh signing ---
# Two independent rerandomizations should be indistinguishable from two
# independent fresh signings. We check both give valid signatures on m
# whose sigma_1 components are independent uniform-looking elements.
t2 = rand()
sigma_p2 = (multiply(sigma[0], t2), multiply(sigma[1], t2))
assert not eq(sigma_p[0], sigma_p2[0]), "distinct rerandomizations collide"
lhs2 = pairing(add(X, multiply(Y, m)), sigma_p2[0])
rhs2 = pairing(G2, sigma_p2[1])
assert lhs2 == rhs2
print("Theorem 2 (unlinkability):          PASS (two rerandomizations both verify, distinct)")

# --- Theorem 1 guard: the trivial signature (t = 0) is degenerate ---
zero_sig = (bc.Z1, bc.Z1)
lhs0 = pairing(add(X, multiply(Y, m)), zero_sig[0]) # pairing with 0 -> 1_GT
rhs0 = pairing(G2, zero_sig[1]) # pairing with 0 -> 1_GT
# The pairing equation holds for ANY m -- this is why the guard matters
assert lhs0 == rhs0
# But the registration contract rejects it because sigma'_1 == 0:
rejected = eq(zero_sig[0], bc.Z1)
assert rejected
print("Theorem 1 guard (sigma'_1 != 0):    PASS (trivial signature rejected by guard)")
```

Theorem 1 (completeness): PASS

Theorem 2 (unlinkability):	PASS (two rerandomizations both verify, distinct)
Theorem 1 guard ($\sigma'_1 \neq 0$):	PASS (trivial signature rejected by guard)

How to read the output: Three PASS lines. The first confirms Theorem 1's pairing identity on a random rerandomization. The second confirms two fresh rerandomizations are distinct and both verify (a minimal demonstration of the unlinkable-yet-valid property). The third demonstrates *why* the $\sigma'_1 \neq 0$ check is mandatory: the pairing equation is trivially satisfied by (O, O) for any m , so only the explicit guard prevents this degenerate signature from passing.

DRAFT

3 Part II: Chaum-Pedersen Re-Encryption Correctness

3.1 Setup

Alice holds a credential whose ElGamal ciphertext is registered on chain and was verified at registration time against her PS signature:

$$E_{\text{alice}} = (R_a, C_a) = (r_a \cdot G, M + r_a \cdot pk_{\text{alice}}), \quad pk_{\text{alice}} = sk_{\text{alice}} \cdot G.$$

To identify herself to Bob she re-encrypts under pk_{bob} with fresh randomness r_b :

$$E_{\text{bob}} = (R_b, C_b) = (r_b \cdot G, M + r_b \cdot pk_{\text{bob}}).$$

She then produces a non-interactive Chaum-Pedersen proof that E_{alice} and E_{bob} encrypt the *same* message point M – without revealing M , m , sk_{alice} , or r_b . The BUCK contract reads E_{alice} from storage (not calldata) and verifies the proof.

3.2 The Statement

Alice proves knowledge of (sk_{alice}, r_b) satisfying the three simultaneous relations

$$pk_{\text{alice}} = sk_{\text{alice}} \cdot G \tag{S1}$$

$$R_b = r_b \cdot G \tag{S2}$$

$$C_a - sk_{\text{alice}} \cdot R_a = C_b - r_b \cdot pk_{\text{bob}}. \tag{S3}$$

(S1) binds her to the registered public key; (S2) binds r_b to the ciphertext she submits; (S3) states that the ElGamal-decrypt of E_{alice} equals the ElGamal-decrypt of E_{bob} under their respective secret keys.

3.3 Protocol (Fiat-Shamir)

Commit. Prover samples $k_1, k_2 \leftarrow \mathbb{Z}_q$ and computes

$$T_1 = k_1 \cdot G, \quad T_2 = k_2 \cdot G, \quad T_3 = k_2 \cdot pk_{\text{bob}} - k_1 \cdot R_a.$$

Challenge.

$$e = H(G, pk_{\text{alice}}, pk_{\text{bob}}, R_a, C_a, R_b, C_b, T_1, T_2, T_3, \text{msg.sender}, \text{spender}, \text{chainid}).$$

$$\textbf{Respond. } s_1 = k_1 - e \cdot sk_{\text{alice}} \pmod{q}, \quad s_2 = k_2 - e \cdot r_b \pmod{q}.$$

Verify. The contract recomputes e from the public inputs and the prover-supplied commitments, then checks

$$s_1 \cdot G + e \cdot pk_{\text{alice}} \stackrel{?}{=} T_1 \tag{V1}$$

$$s_2 \cdot G + e \cdot R_b \stackrel{?}{=} T_2 \tag{V2}$$

$$s_2 \cdot pk_{\text{bob}} - s_1 \cdot R_a + e \cdot (C_b - C_a) \stackrel{?}{=} T_3. \tag{V3}$$

3.4 Theorem 3 (Completeness)

Statement. An honestly generated transcript satisfies (V1), (V2), (V3).

Proof. (V1) and (V2) are standard Schnorr identities:

$$\begin{aligned} s_1 G + e \cdot pk_{\text{alice}} &= (k_1 - e \cdot sk_{\text{alice}})G + e \cdot sk_{\text{alice}}G = k_1 G = T_1, \\ s_2 G + e \cdot R_b &= (k_2 - er_b)G + er_b G = k_2 G = T_2. \end{aligned}$$

For (V3), substitute s_1, s_2 and rearrange:

$$\begin{aligned} s_2 \cdot pk_{\text{bob}} - s_1 \cdot R_a + e(C_b - C_a) &= (k_2 - er_b)pk_{\text{bob}} - (k_1 - e \cdot sk_{\text{alice}})R_a + e(C_b - C_a) \\ &= [k_2 pk_{\text{bob}} - k_1 R_a] + e[sk_{\text{alice}}R_a - r_b \cdot pk_{\text{bob}} + C_b - C_a] \\ &= T_3 + e[-(C_a - sk_{\text{alice}}R_a) + (C_b - r_b \cdot pk_{\text{bob}})] \\ &= T_3 + e \cdot [-M + M] \\ &= T_3. \end{aligned}$$

where the penultimate line uses the witness relations from the honest setup, and the final line uses (S3). ■

3.5 Theorem 4 (Special Soundness / 2-Extractability)

Statement. Given two accepting transcripts $(T_1, T_2, T_3, e, s_1, s_2)$ and $(T_1, T_2, T_3, e', s'_1, s'_2)$ with $e \neq e'$ (and identical public inputs), there is an efficient algorithm that outputs a witness (sk^*, r^*) satisfying (S1), (S2), (S3).

Proof (extractor). Subtract (V1) for the two transcripts:

$$(s_1 - s'_1)G + (e - e') \cdot pk_{\text{alice}} = O \implies pk_{\text{alice}} = \frac{s'_1 - s_1}{e - e'} \cdot G.$$

Set $sk^* := (s'_1 - s_1)(e - e')^{-1} \bmod q$; this is well-defined because q is prime and $e \neq e'$. By construction (S1) holds. Similarly from (V2), $r^* := (s'_2 - s_2)(e - e')^{-1} \bmod q$ satisfies (S2): $R_b = r^* G$.

For (S3), subtract (V3) for the two transcripts:

$$(s_2 - s'_2) \cdot pk_{\text{bob}} - (s_1 - s'_1) \cdot R_a + (e - e')(C_b - C_a) = O.$$

Substitute $s_2 - s'_2 = -(e - e')r^*$ and $s_1 - s'_1 = -(e - e')sk^*$:

$$-(e - e')r^* \cdot pk_{\text{bob}} + (e - e')sk^* \cdot R_a + (e - e')(C_b - C_a) = O.$$

Dividing through by the nonzero scalar $(e - e')$:

$$sk^* \cdot R_a - r^* \cdot pk_{\text{bob}} + (C_b - C_a) = O \iff C_a - sk^* \cdot R_a = C_b - r^* \cdot pk_{\text{bob}},$$

which is (S3). Therefore (sk^*, r^*) is a valid witness, and the extractor runs in the time of four field inversions and a constant number of group operations. ■

Remark (implication for cheating provers). Special soundness lifts, via the forking lemma², to a reduction from any efficient prover that convinces the verifier on a false statement to an efficient solver for discrete logarithms on G_1 . Under the DLog assumption on alt_bn128 , no such prover exists. In particular, Alice cannot produce a valid proof for E_{bob} encrypting any $M' \neq M$ – not merely "hard to compute," but infeasible under a standard assumption Ethereum already requires.

3.6 Theorem 5 (Honest-Verifier Zero-Knowledge)

Statement. There is an efficient simulator $\mathcal{S}$ that, given only the public inputs and a challenge e drawn from the verifier's distribution, outputs a transcript $(T_1, T_2, T_3, e, s_1, s_2)$ distributed *identically* to an honest prover's transcript conditioned on that challenge.

Proof (simulator). $\mathcal{S}$ samples $s_1, s_2 \leftarrow \mathbb{Z}_q$ uniformly, then *defines*

$$\begin{aligned} T_1 &:= s_1 \cdot G + e \cdot pk_{\text{alice}}, \\ T_2 &:= s_2 \cdot G + e \cdot R_b, \\ T_3 &:= s_2 \cdot pk_{\text{bob}} - s_1 \cdot R_a + e \cdot (C_b - C_a). \end{aligned}$$

Equations (V1), (V2), (V3) hold by construction, so the transcript is accepting.

Distribution match. In the real protocol, k_1, k_2 are uniform on $\mathbb{Z}_q$; the honest responses $s_i = k_i - e \cdot w_i$ are therefore uniform on $\mathbb{Z}_q$ (shift by a constant). The commitments T_1, T_2, T_3 are *deterministic functions* of the s_i and the public inputs: (V1), (V2), (V3) each uniquely determine one T_i from the corresponding equation. The simulator samples s_i uniformly and computes the T_i via the same equations – the joint distribution over $(T_1, T_2, T_3, s_1, s_2)$ is identical. ■

Corollary (NIZK via Fiat-Shamir in the ROM). Model H as a programmable random oracle. The Fiat-Shamir transform replaces the verifier's challenge with $e = H(\text{public inputs}, T_1, T_2, T_3, \text{context})$. Standard arguments³ promote:

- HVZK $\Rightarrow$ NIZK (the simulator programs H so that $H(\text{public inputs}, T_1, T_2, T_3, \dots) = e$ for the (T_i) it produced);
- special soundness $\Rightarrow$ proof-of-knowledge soundness (the forking lemma rewinds the oracle).

The binding of `msg.sender`, `spender`, `chainid` into the hash input is standard domain separation that prevents cross-context replay.

3.7 Sanity Check (py_ecc on BN254)

```
# Build registered credential E_alice for Alice
sk_alice = rand()
pk_alice = multiply(G1, sk_alice)
r_a      = rand()
R_a      = multiply(G1, r_a)
```

²Pointcheval, D. and Stern, J. "Security Arguments for Digital Signatures and Blind Signatures." Journal of Cryptology 13(3),

1. The forking lemma: an adversary that convinces a sigma-protocol verifier with non-negligible probability can be rewound to produce two accepting transcripts with distinct challenges, from which the witness is extracted.

³Fiat, A. and Shamir, A. "How to Prove Yourself: Practical Solutions to Identification and Signature Problems." CRYPTO 1986. The transform replacing an interactive verifier's challenge with a hash of the transcript, analyzed in the random oracle model of Bellare and Rogaway (CCS 1993).

```

M          = multiply(G1, m)                                # identity point M = m * G
C_a        = add(M, multiply(pk_alice, r_a))

# Bob's identity public key (Alice knows this from the registry)
sk_bob = rand()
pk_bob = multiply(G1, sk_bob)

# Re-encryption for Bob: E_bob = (r_b * G, M + r_b * pk_bob)
r_b = rand()
R_b = multiply(G1, r_b)
C_b = add(M, multiply(pk_bob, r_b))

def cp_prove(k1, k2):
    T1 = multiply(G1, k1)
    T2 = multiply(G1, k2)
    T3 = add(multiply(pk_bob, k2), neg(multiply(R_a, k1)))
    tag = "|".join(str(v) for v in
                    [G1, pk_alice, pk_bob, R_a, C_a, R_b, C_b, T1, T2, T3])
    e = int(hashlib.sha256(tag.encode()).hexdigest(), 16) % ORDER
    s1 = (k1 - e * sk_alice) % ORDER
    s2 = (k2 - e * r_b) % ORDER
    return (T1, T2, T3, e, s1, s2)

def cp_verify(T1, T2, T3, e, s1, s2):
    v1 = eq(add(multiply(G1, s1), multiply(pk_alice, e)), T1)
    v2 = eq(add(multiply(G1, s2), multiply(R_b, e)), T2)
    Cdiff = add(C_b, neg(C_a))
    v3 = eq(add(add(multiply(pk_bob, s2), neg(multiply(R_a, s1))),
                multiply(Cdiff, e)), T3)
    return v1, v2, v3

# --- Theorem 3: completeness ---
proof = cp_prove(rand(), rand())
v1, v2, v3 = cp_verify(*proof)
assert v1 and v2 and v3, "Theorem 3 FAILED"
print("Theorem 3 (completeness):          PASS")

# --- Theorem 4: extract witness from two transcripts with same (T1,T2,T3) ---
# We fix (k1, k2) to produce the same (T1, T2, T3), then force two
# different challenges by hashing with two different context strings
# (in production the different challenges arise from rewinding the R0).
k1, k2 = rand(), rand()
T1 = multiply(G1, k1)
T2 = multiply(G1, k2)
T3 = add(multiply(pk_bob, k2), neg(multiply(R_a, k1)))

def responses(e):
    return (k1 - e * sk_alice) % ORDER, (k2 - e * r_b) % ORDER

e_1 = rand()
e_2 = rand()
while e_1 == e_2:
    e_2 = rand()
s1_a, s2_a = responses(e_1)
s1_b, s2_b = responses(e_2)

def inv_mod(a, p):
    return pow(a, -1, p)

delta_e = (e_1 - e_2) % ORDER
delta_e_inv = inv_mod(delta_e, ORDER)
sk_star = ((s1_b - s1_a) * delta_e_inv) % ORDER # should equal sk_alice
r_star = ((s2_b - s2_a) * delta_e_inv) % ORDER # should equal r_b

assert sk_star == sk_alice, "Theorem 4 sk extraction FAILED"
assert r_star == r_b, "Theorem 4 r extraction FAILED"
# Witnesses satisfy (S1), (S2), (S3):
assert eq(multiply(G1, sk_star), pk_alice)

```

```

assert eq(multiply(G1, r_star), R_b)
assert eq(add(C_a, neg(multiply(R_a, sk_star))),
          add(C_b, neg(multiply(pk_bob, r_star))))
print("Theorem 4 (special soundness):          PASS (sk, r extracted from two transcripts)")

# --- Theorem 5: simulator produces accepting transcript without witness ---
def cp_simulate(e):
    s1 = rand(); s2 = rand()
    T1 = add(multiply(G1, s1), multiply(pk_alice, e))
    T2 = add(multiply(G1, s2), multiply(R_b, e))
    Cdiff = add(C_b, neg(C_a))
    T3 = add(add(multiply(pk_bob, s2), neg(multiply(R_a, s1))),
             multiply(Cdiff, e))
    return (T1, T2, T3, e, s1, s2)

sim = cp_simulate(rand())
v1, v2, v3 = cp_verify(*sim)
assert v1 and v2 and v3, "Theorem 5 FAILED (simulated transcript rejected)"
print("Theorem 5 (HVZK):                      PASS (simulator accepts without witness)")

# --- Unsoundness counter-example: wrong M at destination ---
M_fake = multiply(G1, rand())
C_b_fake = add(M_fake, multiply(pk_bob, r_b))
# Honest prover tries to produce a proof for (E_alice, (R_b, C_b_fake))
k1, k2 = rand(), rand()
T1 = multiply(G1, k1)
T2 = multiply(G1, k2)
T3 = add(multiply(pk_bob, k2), neg(multiply(R_a, k1)))
tag = "|".join(str(v) for v in
               [G1, pk_alice, pk_bob, R_a, C_a, R_b, C_b_fake, T1, T2, T3])
e_f = int(hashlib.sha256(tag.encode()).hexdigest(), 16) % ORDER
s1_f = (k1 - e_f * sk_alice) % ORDER
s2_f = (k2 - e_f * r_b) % ORDER
v1_f = eq(add(multiply(G1, s1_f), multiply(pk_alice, e_f)), T1)
v2_f = eq(add(multiply(G1, s2_f), multiply(R_b, e_f)), T2)
Cdiff_f = add(C_b_fake, neg(C_a))
v3_f = eq(add(add(multiply(pk_bob, s2_f), neg(multiply(R_a, s1_f))),
             multiply(Cdiff_f, e_f)), T3)
assert v1_f and v2_f and not v3_f, "Unsoundness: fake M proof should fail at V3"
print("Counter-example (M != M'):            V1,V2 PASS; V3 FAIL (as required)")

Theorem 3 (completeness):          PASS
Theorem 4 (special soundness):     PASS (sk, r extracted from two transcripts)
Theorem 5 (HVZK):                 PASS (simulator accepts without witness)
Counter-example (M != M'):         V1,V2 PASS; V3 FAIL (as required)

```

How to read the output: Four PASS lines plus the counter-example. The counter-example is the mirror image of Theorem 4: when the statement is *false* (different message points), the first two verification equations still pass (Alice's key and randomness are genuine) but (V3) – the "same-message" equation – fails. This is precisely the mathematical impossibility the prose in the Identity document refers to when it says "no valid proof exists."

4 Part III: End-to-End Re-Encryption Correctness

4.1 Chain of Trust

The on-chain identity pipeline chains together the two primitives proved above:

KYC ceremony:	issuer signs m	-->	σ on m	(q-SDH)
Fountain:	rerandomize σ	-->	σ' on same m	(Theorem 1, 2)
Registration:	PS + NIZK check	-->	E_{alice} bound to same m	(PS verify + NIZK soundness)
approve(Bob):	Alice re-encrypts	-->	E_{bob}	(Theorem 3)
	Chaum-Pedersen	-->	verified on chain	(Theorems 3, 4, 5)
Decryption:	Bob computes M	-->	$M = C_b - sk_{\text{bob}} * R_b$	(ElGamal correctness)
Two-channel:	Bob receives	-->	$H(\text{identity_data}) * G == M$	(hash commitment)

4.2 Theorem 6 (End-to-End Re-Encryption Correctness)

Statement. Suppose the BUCK contract:

1. holds a registered credential $E_{\text{alice}} = (R_a, C_a)$ together with pk_{alice} and a PS signature σ' that verified at registration time against a trusted issuer public key (X, Y) ;
2. accepts a Chaum-Pedersen proof π supplied by `msg.sender` = Alice, with spender pk_{bob} and submitted ciphertext $E_{\text{bob}} = (R_b, C_b)$.

Then, under the DLog assumption on G_1 and in the ROM, there exists $M = m \cdot G$ such that

$$C_a - sk_{\text{alice}} \cdot R_a = M = C_b - sk_{\text{bob}} \cdot R_b,$$

and the scalar m is the issuer-certified identity scalar covered by σ' .

Proof sketch. The registration check evaluates $e(\sigma'_1, X + mY) = e(\sigma'_2, g_2)$ together with a NIZK that binds the same m to E_{alice} (so $C_a - sk_{\text{alice}} R_a = m \cdot G$); existential unforgeability of PS under q-SDH ensures m is the scalar the issuer signed. The contract reads E_{alice} from storage, not calldata, so the value is frozen.

Theorem 4 (special soundness) plus the forking lemma reduction to DLog gives: from any adversary producing an accepting π with non-negligible probability there is an extractor outputting witnesses (sk^*, r^*) with $C_a - sk^* R_a = C_b - r^* pk_{\text{bob}}$. Since $pk_{\text{alice}} = sk^* \cdot G = sk_{\text{alice}} \cdot G$ and G is a generator, $sk^* = sk_{\text{alice}}$; so both sides equal the registered M . Therefore E_{bob} decrypts under sk_{bob} to the same M that the issuer's PS signature attests to. ■

4.3 Why This Suffices for Every Downstream Use Case

- **approve / transfer / transferFrom (Identity):** Bob decrypts E_{bob} and obtains M ; combined with off-chain `identity_data` delivery and the check $H(\text{identity_data}) \cdot G = M$, he learns Alice's real identity with cryptographic assurance. A BUCK **transfer** / **transferFrom** is gated on this **approve** handshake against *both* parties' registered records, so a value move carries an account $\leftrightarrow$ Identity binding for *each* counterparty – the bilateral mutual decryptability formalised in the EOA mirror below.
- **Notes spend predicates (Part IV):** The same re-encryption / decryption algebra underlies the Identity-M BUCK Notes. At mint the issuer addresses the recipient *Identity point* M_{rec} and a mint-binding forces the ciphertext over the issuer's *own* registered M_{iss} (Theorem 7); at

spend the deposit-coupling sigma is the Chaum-Pedersen statement of this Part, lifted inside a SNARK – so Theorem 4’s special soundness lifts directly and binds the depositing account to the registered Identity scalar m_{rec} , with the note ciphertext decrypting to a *registered* member (Theorem 8); the bearer B1 path instead binds the depositor’s *own* Identity M_{dep} for the public issuer (Theorem 9). Unlike the earlier account-pinned design, the binding is to the *Identity*, not a mint-time keypair – *recovery*, *not finality* on key loss – yet identity laundering through the note pool remains infeasible.

- **Identity Guardians (approveFor, recovery, cosigning):** The Schnorr proof of knowledge of sk is a one-statement specialization of the above (only (S1) with a Fiat-Shamir binding to `(msg.sender, principal)`). Theorems 3-5 apply verbatim with the second and third statements dropped.

4.4 The EOA Transfer Mirror (`transfer` / `transferFrom`)

A private $\rightarrow$ private BUCK **transfer** achieves the *same* mutual-decryptability invariant as a Notes spend, but *without* a SNARK and with collusion-resistance for free – precisely because the transfer is *not* anonymous: both counterparties are named at one moment. The bilateral **approve** handshake reads *both* registered records in a single call and runs the re-encryption of this Part in each direction; the value move is gated on it.

Step	Cryptographic step	Who is named	Guarantee
register	store $(pk, E_{\text{addr}} = \text{Enc}(M, pk)) + \text{PS credential}$	–	Identity $\leftrightarrow$ KYC (Theorem 4)
approve (bilateral)	re-encrypt $E_{A \rightarrow B} = \text{Enc}(M_A, pk_B) + \text{Chaum-Pedersen} \equiv E_{\text{addr}}[A]$	both	sound re-encryption under pk_B
transfer / transferFrom	value $A \rightarrow B$, gated on the approve	both (linked)	mutual decryptability established
receipt	verifiable_decrypt $E_{X \rightarrow \text{me}} \rightarrow M$	–	either party names the other

Corollary (EOA transfer mutual decryptability). For a **transfer** / **transferFrom** gated on bilateral **approve** proofs, each counterparty can recover the other’s registered Identity point (B decrypts $E_{A \rightarrow B}$ under sk_B to M_A ; A decrypts $E_{B \rightarrow A}$ under sk_A to M_B), each can prove it under compulsion (verifiable decryption), and no third party lacking either secret key recovers M_A or M_B (ElGamal IND-CPA, hence DDH). This is the BUCK mutual-decryptability invariant in its *cheapest* form; the Identity-M Notes (Part IV) reconstruct the *same* invariant across an anonymising mint $\rightarrow$ spend detour, which is exactly what forces the extra apparatus there (Theorems 7-11).

Proof. Each direction is Theorem 6 applied to the respective ordered pair: an accepted Chaum-Pedersen proof certifies that $E_{A \rightarrow B}$ re-encrypts A ’s registered, PS-certified M_A under pk_B , so $C_{A \rightarrow B} - sk_B R_{A \rightarrow B} = M_A$ by ElGamal correctness; symmetrically for $B \rightarrow A$. Third-party secrecy is the IND-CPA security of ElGamal on G_1 : deciding whether a ciphertext encrypts a candidate M is a DDH decision (see *Harvest-Now, Decrypt-Later* for the quantum caveat). ■

4.5 Sanity Check (py_ecc on BN254)

```
# Reuse M (Alice’s identity M_A), (sk_alice, pk_alice), (sk_bob, pk_bob),
# and Alice’s re-encryption to Bob E_bob = (R_b, C_b) from Part II.
def decrypt(R, C, sk): return add(C, neg(multiply(R, sk))) # ElGamal decrypt
def reencrypt(P, pk): # Enc(P, pk)
    r = rand(); return (multiply(G1, r), add(P, multiply(pk, r)))

M_A = M # Alice’s Identity point
m_bob = int(hashlib.sha256(b"Bob Smith | Alberta | AIC-2026").hexdigest(), 16) % ORDER
M_B = multiply(G1, m_bob) # Bob’s Identity point

# approve(Alice -> Bob): Bob decrypts Alice’s re-encryption to M_A
assert eq(decrypt(R_b, C_b, sk_bob), M_A)
```

```

# approve(Bob -> Alice): Alice decrypts Bob's re-encryption to M_B
R_ba, C_ba = reencrypt(M_B, pk_alice)
assert eq(decrypt(R_ba, C_ba, sk_alice), M_B)
print("Corollary (transfer mutual decryptability):          PASS")

# Mallory holds neither sk: a wrong key recovers a wrong point (DDH)
assert not eq(decrypt(R_b, C_b, sk_alice), M_A)
assert not eq(decrypt(R_ba, C_ba, sk_bob), M_B)
print("Corollary (third-party cannot decrypt):              PASS")

Corollary (transfer mutual decryptability):                PASS
Corollary (third-party cannot decrypt):                     PASS

```

DRAFT

5 Part IV: BUCK Notes Spend Predicates (A1, A2, B1)

5.1 Setup

Pool. `noteTree`, a depth-20 Poseidon tree of leaves $cm = \text{Poseidon}_5(\text{flavor}, v, \rho, \text{idHash}, \text{predicate})$ (`idHash` digests the per-flavor identity payload); a `nullifiers` set; a 30-deep ring of recent roots. Mint is the batch rollup of *alberta-buck-notes-rollup-mint* – a Groth16 proof of incremental insertion of N fresh leaves with in-circuit range checks ($v_i, \text{totalFace} \in [0, 2^{128})$) and value conservation ($\sum_i v_i = \text{totalFace}$); the contract pins `oldRoot` / `nextLeafIndex` and pulls `totalFace` BUCK. Demurrage is orthogonal.

Identity accumulator. IdentityRegistry maintains a *second* Poseidon tree whose leaves are registered Identity points $\ell(M) = \text{Poseidon}(M.x, M.y)$, with root `rtid` on chain (updated per `register` / `bindContract`, commit-before-use). Membership of M is the statement " M is a genuine KYC-registered Identity" – the load-bearing addition over the earlier account-pinned spend, and the half that makes a colluding counterparty *un-spendable* rather than merely un-nameable.

Identity primitives (recap, Parts I-III). A registered account is (`addr`, $pk = sk\ G$, $E_{\text{addr}} = (rG, M + r\ pk)$): an ElGamal encryption of its issuer-certified Identity point $M = mG$ under its own key (PS-signed at registration). The Identity Fountain re-issues unlimited fresh accounts ($sk_i, pk_i, E_{\text{addr},i}$) on the *same* M .

Note addressing (mint). The issuer addresses the recipient *Identity* $M_{\text{rec}} = m_{\text{rec}}G$, not an account, through a single ciphertext the recipient's identity scalar m_{rec} decrypts (and that scalar is shared by every Fountain account of the recipient):

$$A2: eIss = (r'G, M_{\text{iss}} + r'M_{\text{rec}}), \quad A1: eRec = (r'G, M_{\text{rec}} + r'M_{\text{rec}}).$$

A2 hides the issuer (recovered only by the recipient); A1 names it in the clear (m_{iss} plus a Schnorr σ_{iss} over the batch `keccak(cms)`). B1 is bearer – no recipient ciphertext at mint, the public issuer signs the batch – and the depositor encrypts *its own* Identity for the issuer at spend (below). A mint-binding (a Schnorr for public issuers, a recipient-blinded re-encryption proof for A2) forces the ciphertext over the issuer's *own* registered Identity, so no victim can be framed (Theorem 7).

Nullifier. Flavor-agnostic, over the note proof's opening with a domain tag: $nf = \text{Poseidon}_3(\rho, \text{idHash}, \text{tag})$.

5.2 Theorem 7 (Mint Addressing and Anti-Framing)

Statement. Let the recipient have Identity $M_{\text{rec}} = m_{\text{rec}}G$ and the issuer a registered credential $E_{\text{iss}}^{\text{reg}}$ decrypting (under sk_{iss}) to its certified M_{iss} . For fresh r' :

1. **(Decryptability.)** $eIss = (r'G, M_{\text{iss}} + r'M_{\text{rec}})$ decrypts under m_{rec} to M_{iss} , and $eRec = (r'G, M_{\text{rec}} + r'M_{\text{rec}})$ decrypts to M_{rec} – so *any* Fountain account of the recipient can open it.
2. **(Anti-framing.)** An accepted mint-binding – `verifyIssuerSchnorr` (public issuer) or the recipient-blinded re-encryption `verifyIssuerReenc` (A2) – forces the encrypted point to be the issuer's *own* registered M_{iss} , read from $E_{\text{iss}}^{\text{reg}}$ at `msg.sender`. No victim's Identity can be substituted.
3. **(Issuer ignorance + unlinkability.)** The issuer knows only the public point M_{rec} (revealed off chain), never m_{rec} ; fresh r' makes two mints to the same recipient IND-CPA-indistinguishable.

Proof. (1) ElGamal correctness with public key $M_{\text{rec}} = m_{\text{rec}}G$: $C - m_{\text{rec}}R = M + r'M_{\text{rec}} - m_{\text{rec}}r'G = M$, since $m_{\text{rec}}G = M_{\text{rec}}$. (2) `verifyIssuerSchnorr` is EUF-CMA (Theorem 9 citation);

`verifyIssuerReenc` is a recipient-blinded Okamoto sigma whose special soundness (Theorem 4 shape) extracts (sk_{iss}, r') binding the ciphertext to $E_{\text{iss}}^{\text{reg}}$'s plaintext – a substituted point breaks it. (3) The encryption manipulates group elements only; recovering M_{rec} or m_{rec} is ElGamal IND-CPA / DLog. ■

Remark (no re-randomization needed). The earlier account-pinned design re-randomised the recipient's *account* credential, binding the note to one keypair. Identity-M addresses the recipient *point* instead, so the note is redeemable by any account on that Identity – *recovery, not finality* on key loss (Theorem 8).

5.3 Theorem 8 (Deposit-Coupling Soundness – A1, A2)

Statement. Suppose the contract accepts an addressed spend (`spendCoupledA1` / `spendCoupledA2`): the note proof, the deposit-coupling sigma `verifyDepositCoupling(depositor, eEnc, dc)`, and the membership proof of `dc.P_I`. Then under SNARK knowledge-soundness, special soundness of the Okamoto sigma (Theorem 4 shape), DLog on G_1 , and the ROM, there is an extractor producing $(m_{\text{rec}}, sk_{\text{dep}}, b)$ and a Merkle path such that, with the depositor's registered $E_{\text{dep}} = (R_d, C_d)$ and key pk_{dep} :

1. cm lies under `noteRoot`; $nf = \text{Poseidon}_3(\rho, idHash, tag)$.
2. **(Account bound to the Identity.)** $pk_{\text{dep}} = sk_{\text{dep}}G$ and $C_d = m_{\text{rec}}G + sk_{\text{dep}}R_d$: the depositing account decrypts to $M_{\text{rec}} = m_{\text{rec}}G$, its registered, PS-certified Identity (Theorem 6).
3. **(Decryption to the committed point.)** $eEnc$ decrypts under m_{rec} to M^* , with $dc.P_I = M^* + bH$.
4. **(Registered.)** $\ell(M^*) \in \text{rt}_{\text{id}}$.

Here $M^* = M_{\text{iss}}$ (A2: the recovered private issuer) or $M^* = M_{\text{rec}}$ (A1: the recipient itself).

Proof. Two accepting sigma transcripts sharing commitments extract $(m_{\text{rec}}, sk_{\text{dep}}, b)$ satisfying relations E4/E2/E3 (Theorem 4). SNARK extraction gives the Merkle path and the membership witness. The contract feeds `dc.P_I` – not prover-supplied bytes – as the membership public input, so (3) and (4) concern the *same* point. ■

Implications.

- **Recovery, not finality.** The spend is keyed to the *Identity scalar* m_{rec} , not a mint-time keypair. Any registered Fountain account bound to m_{rec} deposits; loss of one account key is recovered by re-registering a fresh account on the same M_{rec} – the opposite of the earlier account-pinned A-spend.
- **Collusion closed (A2).** If a colluding issuer keys $eIss$ to a throwaway point, M^* is a non-member: no membership proof exists (4 fails) and the deposit *reverts*. No spendable-but-unnamed note can be created. (Clause (3) concerns the *eEnc supplied at spend*; Theorem 12 ties that $eEnc$ to the ciphertext the spent note committed, closing the substitute- $eEnc$ escape.)
- **Issuer cannot self-redeem.** The issuer holds M_{rec} but not m_{rec} ; it cannot satisfy (2) for any account it controls – unless it *is* the recipient (self-wormhole).
- **No third-party leakage.** E2/E3 are HVZK (Theorem 5): m_{rec} , M_{rec} , and M^* (blinded in $dc.P_I = M^* + bH$) never appear on chain.

5.4 Theorem 9 (Depositor-Binding Soundness – B1)

Statement. Suppose the contract accepts a bearer spend (`spendCoupledB1`): the note proof, `verifyDepositorBinding(depositor, issuer, eDepForIss, pi)`, and the membership proof of $pi.P_{\text{dep}}$. Then under the same assumptions plus Schnorr EUF-CMA⁴, an extractor produces $(m_{\text{dep}}, sk_{\text{dep}}, r, b)$ such that:

1. $cm = \text{Poseidon}_5(B1, v, \rho, idHash, \cdot)$ lies under `noteRoot`, and the prover knows ρ (the bearer authorization).
2. (**Account bound to the Identity.**) $pk_{\text{dep}} = sk_{\text{dep}}G$, $C_d = m_{\text{dep}}G + sk_{\text{dep}}R_d$.
3. (**Encrypted for the issuer.**) $eDepForIss = (rG, m_{\text{dep}}G + rpk_{\text{iss}})$ – the depositor’s M_{dep} under the public issuer’s key, the *same* m_{dep} as (2).
4. (**Committed and registered.**) $P_{\text{dep}} = m_{\text{dep}}G + bH$ and $\ell(M_{\text{dep}}) \in \text{rt}_{\text{id}}$.
5. The public issuer is named by m_{iss} plus an EUF-CMA Schnorr over the batch.

Proof. The five-relation Okamoto sigma extracts $(m_{\text{dep}}, sk_{\text{dep}}, r, b)$ (Theorem 4); the single response for m_{dep} couples (2), (3), (4) to one Identity; the contract feeds P_{dep} to the membership verifier, so (4)’s clauses are the same point. Schnorr EUF-CMA gives (5). ■

Implications.

- **The public issuer names the bearer depositor.** From the `SpentCoupledB1` event the issuer decrypts $eDepForIss$ with sk_{iss} to recover M_{dep} – collusion-resistantly (a substituted ciphertext breaks (3)) and privately from third parties (IND-CPA).
- **Collusion closed.** A bogus M_{dep} is a non-member: (4) fails and the deposit reverts. No one cashes a B1 note un-nameably.
- **Bearer race is civil-trust.** Whoever publishes ρ first wins; the issuer who chose ρ can race but is identified at the moment of theft. The cryptography places no prior restraint – the legal system does, exactly as for a paper cashier’s cheque.

5.5 Theorem 10 (Nullifier and Double-Spend Prevention)

The nullifier is flavor-agnostic, $nf = \text{Poseidon}_3(\rho, idHash, \text{tag})$, over the note proof’s opening ($idHash$ digests the identity payload; the tag separates A- from B-shape so one `nullifiers` set serves both).

1. (**Determinism / double-spend.**) nf is a deterministic function of $(\rho, idHash)$; two accepting spends of the same cm yield the same nf , and the contract’s $nf \notin \text{nullifiers}$ check rejects the second.

⁴Schnorr, C.-P. "Efficient Signature Generation by Smart Cards." Journal of Cryptology 4(3), 1991, pp. 161-174. The Schnorr signature scheme is EUF-CMA secure under the discrete logarithm assumption in the random oracle model; see Pointcheval and Stern (2000) ² for the forking-lemma reduction, and Bellare and Neven, "Multi-signatures in the Plain Public-Key Model" (CCS 2006) for tightened security bounds.

2. **(Third-party unlinkability.)** Modelling Poseidon as a random oracle, nf is a PRF over the secret opening; without $(\rho, idHash) - \rho$ a uniform q -element – inverting nf to its cm is pre-image resistance. So *no party lacking the opening* (in particular Mallory) links a spend to its mint. The two counterparties who hold the opening already know the note exists; the nullifier protects the transaction *graph* from outside observers – exactly what bulk-trace resistance requires.

Proof. (1) is syntactic; (2) is the standard RO/PRF pre-image argument. ■

5.6 Theorem 11 (Mutual Decryptability and Transaction-Graph Privacy)

Statement. For every accepted spend, *both* counterparties can name the other as a *registered* Identity, while no third party can name either or link the mint to the spend.

Recipient names the issuer.

- *A1 (public):* the note carries m_{iss} + an EUF-CMA Schnorr; the recipient reads M_{iss} directly.
- *A2 (private):* the recipient decrypts $eIss$ under m_{rec} to M_{iss} (Theorem 7.1), and Theorem 8’s membership certifies it is registered; anti-framing (Theorem 7.2) forces it to be the true minter, and the note-binding tie (Theorem 12) forces the spend’s verified ciphertext to be the one *this note* committed. This is the close of the recipient-key collusion gap – an issuer cannot produce a *spendable* note the recipient cannot *name*.

Issuer names the depositor.

- *A1, A2 (addressed):* the issuer chose M_{rec} at mint – it holds the symmetric knowledge by construction.
- *B1 (bearer):* the issuer decrypts the $eDepForIss$ in `SpentCoupledB1` under sk_{iss} to M_{dep} (Theorem 9), certified registered by membership.

Both directions resolve to a registered Identity point and – via the off-chain identity_data path (Theorem 6) – the real-world KYC identity. This is the BUCK mutual-decryptability invariant, preserved across the pool detour and *enforced*: the membership half makes an un-nameable counterparty *un-spendable*, so the invariant holds even against a colluding pair, not merely an honest one.

Privacy corollary (what Mallory cannot do). The chain shows an opaque cm , a fresh nf , a public (*face, recipient*), and the deposit ciphertexts ($eIss/eRec/eDepForIss$ and the blinded commitment P) – all IND-CPA or perfectly hiding. Lacking any counterparty secret, Mallory learns: no Identity (third-party non-disclosure), no $cm \leftrightarrow nf$ link (Theorem 10 – transaction-graph privacy), and no link between two notes to the same recipient (Theorem 7.3). Bilateral disclosure between the two parties is preserved; bulk harvest is defeated. ■

5.7 Theorem 12 (Note-Binding Re-Encryption Tie – A1, A2)

Theorems 8 and 11 reason about *the ciphertext the depositor supplies at spend* ($eEnc$). On their own they leave one degree of freedom: nothing yet forces that $eEnc$ to be *about the note being spent*. A depositor holding a stolen opening could attach its *own* self-addressed $eEnc$ (defeating addressed binding), and an A2 coalition could attach a *nameable* substitute for an un-nameable committed $eIss$ (re-opening the collusion gap). The note-binding relation (`circuits/note_binding.circom`, verified behind `INoteBindingVerifier` in the addressed spends) closes both.

Statement. Suppose the contract accepts an addressed spend with the note-binding verifier active: in addition to the note proof, the deposit-coupling sigma, and the membership proof (Theorem 8), the Groth16 binding proof verifies with public inputs *derived on chain* from the caller's nullifier nf , the supplied $eEnc = (R_e, C_e)$, and the sigma's committed point $dc.P_I$ (the adapter decomposes the six point coordinates into 64-bit limbs; the proof bytes carry only the Groth16 triple). Then under Groth16 knowledge-soundness, the ROM (Poseidon), and the independence of the NUMS point H (no known $\log_G H$), an extractor produces $(\rho, idHash, eNote, eIss_0, s, r, m_{rec}, b, M_I)$ such that:

1. **(Same note.)** $nf = \text{Poseidon}_3(\rho, idHash, tag)$ – the very nullifier this spend consumed – and $idHash = \text{Poseidon}_8(eNote, eIss_0)$ – the identity payload the spent note committed at mint.
2. **(Re-encryption equivalence.)** With $eIss_0 = (R_0, C_0)$: $R_e = R_0 + sG$ and $C_e = C_0 + (s \cdot m_{rec})G$ – the supplied $eEnc$ is a re-randomisation of the *committed* ciphertext under $M_{rec} = m_{rec}G$, carrying the same plaintext.
3. **(ElGamal structure.)** $R_0 = rG$ and $C_0 = M_I + (r \cdot m_{rec})G$: the committed ciphertext decrypts under m_{rec} to M_I .
4. **(Committed point.)** $P_I = M_I + bH$ – the same blinded point the coupling sigma opened and the membership proof certified.

Proof. Constraints (1) are the two Poseidon openings; (2)-(4) are fixed-base scalar-multiplication relations checked in-circuit over 4-limb $\mathbb{F}_q$ arithmetic; knowledge-soundness extracts the witnesses. The sigma's extractor (Theorem 8) independently gives (m'_{rec}, sk_{dep}, b') with $eEnc$ decrypting under m'_{rec} to M^* and $P_I = M^* + b'H$. Since re-encryption preserves the plaintext, clause (2)+(3) give $eEnc$ decrypting under m_{rec} to M_I ; equating the two openings of P_I , $M_I + bH = M^* + b'H$. If $(M_I, b) \neq (M^*, b')$ this yields $\log_G H$, contradicting the NUMS assumption. Hence $M_I = M^*$ and the SNARK's decrypted point *is* the point the sigma authenticated and the membership certified. ■

Implications.

- **Addressed binding enforced.** Clause (2) forces the spend's $eEnc$ to re-encrypt the committed $eIss_0$ under the *same* m_{rec} the coupling sigma authenticated (E2: the depositing account's registered Identity scalar). An account holding a stolen A1/A2 opening cannot redeem with a self-addressed $eEnc$: its own $m \neq m_{rec}$ cannot satisfy (2) and (3) for the committed ciphertext. Only the addressed Identity spends – from any of its Fountain accounts.
- **A2 collusion closed against substitution.** Theorem 8's "collusion closed" implication assumed the verified $eEnc$ was about the spent note; Theorem 12 makes that an extractable fact. A coalition minting an un-nameable $eIss$ (keyed to a throwaway point) can no longer attach a nameable substitute: the binding forces M_I to be the committed ciphertext's decryption, the membership then has no satisfying witness, and the deposit reverts. Un-nameable $\Rightarrow$ un-spendable, now against *active substitution*, not merely honest provers.
- **Unlinkability preserved.** The relation is re-encryption *equivalence*, not equality: requiring the literal mint-time $eIss$ (a public input of `Notes.mint`) would link spend to mint and destroy A2 anonymity. The supplied $eEnc$ is a fresh uniform re-randomisation (IND-CPA), the nullifier is a PRF output (Theorem 10), and P_I is perfectly hiding – the spend's public footprint gains no new linkage surface.

- **Grounding.** Theorem 8 clause (3) and Theorem 11's A2 naming clause are stated over the supplied *eEnc*; Theorem 12 is the bridge that makes them statements about *the spent note's* committed ciphertext.

5.8 Sanity Check (py_ecc on BN254)

```
# Reuse ORDER, G1, multiply, add, neg, eq, rand, hashlib, m, sk_alice,
# pk_alice from Parts I-II. Issuer = Alice (m_iss = m, account key
# (sk_alice, pk_alice)); Bob is the recipient, identity scalar m_rec.
def dec(R, C, sk): return add(C, neg(multiply(R, sk))) # ElGamal decrypt
def leaf(M):      return (M[0].n, M[1].n)             # registry-tree leaf

m_iss = m;      M_iss = multiply(G1, m_iss)           # Alice's Identity
m_rec = int(hashlib.sha256(b"Bob | Alberta | AIC-2026").hexdigest(), 16) % ORDER
M_rec = multiply(G1, m_rec)                          # Bob's Identity (addressed)
H = multiply(G1, int(hashlib.sha256(b"AlbertaBuck:H").hexdigest(), 16) % ORDER)
registered = {leaf(M_iss), leaf(M_rec)}              # both KYC-registered

# --- Theorem 7: addressing decrypts to the intended point under m_rec ---
rp = rand()
eIss = (multiply(G1, rp), add(M_iss, multiply(M_rec, rp))) # Enc(M_iss, M_rec)
eRec = (multiply(G1, rp), add(M_rec, multiply(M_rec, rp))) # Enc(M_rec, M_rec)
assert eq(dec(*eIss, m_rec), M_iss) and eq(dec(*eRec, m_rec), M_rec)
print("Theorem 7 (addressing decrypts under m_rec):      PASS")

# --- Theorem 8: deposit coupling recovers a registered member ---
Mstar = dec(*eIss, m_rec)                            # recipient -> issuer
b = rand(); P_I = add(Mstar, multiply(H, b))          # blinded commitment
assert eq(add(P_I, neg(multiply(H, b))), Mstar)       # P_I - bH == Mstar
assert leaf(Mstar) in registered                     # membership
print("Theorem 8 (coupling -> registered member):      PASS")

# --- Theorem 8 negative: issuer cannot self-redeem (lacks m_rec) ---
assert not eq(dec(*eIss, m_iss), M_iss)
print("Theorem 8 (issuer cannot redeem own note):      PASS")

# --- Theorem 8 collusion: bogus eIss decrypts to a NON-member ---
M_bogus = multiply(G1, rand())
eIss_bad = (multiply(G1, rp), add(M_iss, multiply(M_bogus, rp)))
assert leaf(dec(*eIss_bad, m_rec)) not in registered # -> reverts
print("Theorem 8 (collusion bogus eIss un-spendable):  PASS")

# --- Theorem 10: flavor-agnostic nullifier; deterministic ---
def H3(*f): return int(hashlib.sha256("".join(map(str, f)).encode()).hexdigest(), 16) % ORDER
rho = rand(); idHash = H3(eIss[0][0].n, eIss[0][1].n, eIss[1][0].n, eIss[1][1].n)
assert H3(rho, idHash, 4243) == H3(rho, idHash, 4243) # double-spend caught
print("Theorem 10 (nullifier determinism):             PASS")

# --- Theorem 11: B1 issuer names depositor; both members ---
m_dep = m_rec; M_dep = M_rec
rd = rand(); eDep = (multiply(G1, rd), add(M_dep, multiply(pk_alice, rd)))
assert eq(dec(*eDep, sk_alice), M_dep)               # issuer -> depositor
assert leaf(M_iss) in registered and leaf(M_dep) in registered
print("Theorem 11 (mutual decryptability, both members): PASS")

Theorem 7 (addressing decrypts under m_rec):          PASS
Theorem 8 (coupling -> registered member):           PASS
Theorem 8 (issuer cannot redeem own note):           PASS
Theorem 8 (collusion bogus eIss un-spendable):       PASS
Theorem 10 (nullifier determinism):                   PASS
Theorem 11 (mutual decryptability, both members):    PASS
```

How to read. Theorem 7: the note ciphertext decrypts under the recipient's identity scalar m_{rec} to the intended point (issuer for A2, recipient for A1). Theorem 8: the deposit coupling recovers

M^* , which is a registered member; the issuer (lacking m_{rec}) cannot self-redeem, and a colluding *bogus* ciphertext recovers a non-member – so the spend reverts (no key-loss-terminal failure: any account on M_{rec} works). Theorem 10: one flavor-agnostic nullifier, deterministic, so a replay is caught. Theorem 11: the issuer decrypts the deposit ciphertext to the depositor’s Identity, and both parties’ points are registered – mutual decryptability, both ends KYC-bound.

5.9 Worked Example: An Identity-M Spend End to End

Theorems 7-11 in one A2 transfer – Alice issues privately to Bob, who deposits from a Fountain account no one could have pinned at mint. Executable reference: `alberta_buck/wallet/unilateral_a2.py` with `test_unilateral_a2.py` (11) + `UnilateralA2.t.sol` (7) + `NotesCoupledA2.t.sol` (8). The relation trace:

1. **Register** (Part III). Alice and Bob each hold a PS-certified Identity (m, M) and registered accounts $E_{\text{addr}} = (rG, M + rpk)$; both M ’s are leaves of rt_{id} .
2. **Mint** (Theorem 7). Alice reads Bob’s identity point M_{rec} (off chain) and commits $eIss = (r'G, M_{\text{iss}} + r'M_{\text{rec}})$ with a recipient-blinded re-encryption binding (anti-framing); the leaf $cm = \text{Poseidon}_5(A2, v, \rho, \text{Poseidon}(eIss), 0)$ is folded into `noteTree`.
3. **Deliver + verify** (Theorem 11). Bob decrypts $eIss$ under m_{rec} to M_{iss} and checks $\ell(M_{\text{iss}}) \in \text{rt}_{\text{id}}$ *before* accepting the note.
4. **Deposit** (Theorems 8, 10). From *any* account bound to m_{rec} , Bob submits `spendCoupledA2`: the note proof (cm under `noteRoot`, $nf = \text{Poseidon}_3(\rho, idHash, tag)$); the coupling sigma (account $\leftrightarrow m_{\text{rec}}$; $eIss$ decrypts to $dc.P_I = M_{\text{iss}} + bH$); and the membership of $dc.P_I$. The pool pays `face` BUCK and `SpentCoupledA2` publishes the blinded P_I .
5. **Receipt** (Theorem 11). From m_{rec} alone Bob produces a plaintext, third-party-checkable receipt naming both parties (himself M_{rec} , the issuer M_{iss}); the issuer chose M_{rec} at mint, so the disclosure is mutual.

No on-chain value reveals an Identity; mint and spend are unlinkable to Mallory (Theorem 10); and a bogus $eIss$ would have failed step 3 and reverted at step 4 (Theorem 8).

5.10 Mint-Circuit Status: Phase 7-bis Closure of the Earlier Open Questions

An earlier draft of this document (v0.4) flagged three open correctness obligations against the Phase-6 placeholder mint circuit (`circuits/mint.circom`). The Phase 7-bis batch-mint pivot (`circuits/mint_batch.circom`, deployed as per-N pinned variants `mint_batch_n${N}.circom`; see `alberta-buck-notes-rollup-mint.org`) closes all three at the mint layer. This section records the closure for the proof record so a reader cross-checking the prior draft can see which obligations are now discharged in the circuit vs. deferred to the spend circuit.

5.10.1 Closure 1 (Field-Overflow / Value-Conservation Range Check) – *Discharged in circuit*

The Phase-6 draft’s value-conservation constraint $\sum_{i=1}^N v_i === \text{totalFace}$ is an equation in $\mathbb{F}_r$ with $r \approx 2^{254}$ and therefore admits integer-overflow witnesses unless each v_i is bounded. `mint_batch.circom` now applies `Num2Bits(128)` to every v_i and to `totalFace` (constraint family (R) in the circuit

source). With $N \leq 2^{20}$ (the tree-depth cap) and 128-bit per-leaf face, the integer sum cannot overflow: $N \cdot 2^{128} \leq 2^{148} \ll 2^{254} = r$. No spend circuit can be tricked into releasing more BUCK than was escrowed by an integer-vs-field-arithmetic mismatch on the mint side.

5.10.2 Closure 2 (Single transferFrom $\leftrightarrow$ N Commitments) – *Discharged by SNARK binding*

Phase 7-bis makes the `totalFace` public input of the mint SNARK both (i) the value passed to `buck.transferFrom` and (ii) the value the circuit forces to equal $\sum_i v_i$. The single `Transfer` event of size `totalFace` is therefore cryptographically bound to the Groth16 proof's public inputs – which include the entire `cms[]` array on the per-N pinned verifiers (see alberta-buck-notes-rollup-mint.org, "Calldata delivery"). No separate batch-identifier storage slot is needed for soundness; on-chain analytics that want a transfer $\leftrightarrow$ batch correspondence can recover it from the `Minted` event's `cms` parameter and the simultaneous `Transfer`.

5.10.3 Closure 3 (Spend-Circuit Trio) – *Mint half discharged; nullifier + spend-side value conservation deferred to spend.circom*

The earlier draft's three coupled obligations – Merkle membership, nullifier determinism, and value conservation – now split cleanly between the two circuits:

1. **Merkle membership (mint side).** `mint_batch` performs an in-circuit dual Merkle walk per leaf (constraint family (M) in the circuit source): the empty-seat walk constrains every witness sibling against the rolling prior root, and the filled-seat walk advances the rolling root with cm_i inserted. After all N leaves the final rolling root is constrained to equal the public `newRoot`. The contract accepts `newRoot` into its root-history ring buffer; spend circuits prove inclusion against any root in that window. The Phase-2 flat `commitments[]` array is *gone*.
2. **Nullifier determinism (spend side).** The spend circuit recomputes the flavor-agnostic $nf = \text{Poseidon}_3(\rho, idHash, tag)$ from the committed opening (Theorem 10); closure belongs to `circuits/spend.circom`, the one note proof every coupled spend reuses.
3. **Value conservation (spend side).** The spend circuit must still output `faceReleased` equal to the committed v in the opening, with v the same 128-bit-bounded quantity range-checked at mint. The mint-side range check (Closure 1) remains load-bearing here: a spend circuit that trusts v without re-bounding it relies on the mint circuit having already done so.

The mint-circuit layout is now *frozen* at $\text{Poseidon}_5(\text{flavor}, v, \rho, idHash, \text{predicate})$ over BN254 with tree depth 20; spend-circuit design no longer ratifies the mint-circuit layout because the layout is published and pinned by the per-N verifiers already deployed.

6 Security Assumptions Summary

Theorem	Primitive	Assumption needed	Condition
Theorem 1 (PS completeness)	PS	bilinearity of e	no (uncon)
Theorem 2 (PS unlinkability)	PS	prime-order G_1 , uniform t	no (inform)
Theorem 3 (CP completeness)	CP	group axioms	no (uncon)
Theorem 4 (CP special soundness)	CP	none for extractor; DLog for cheating-prover reduction	DLog (only)
Theorem 5 (CP HVZK)	CP	uniform k_1, k_2	no (perfect)
NIZK corollary	CP + FS	ROM	ROM
Theorem 6 (end-to-end)	PS + CP	q-SDH (PS unforgeability), DLog, ROM	yes
Transfer mutual-decryptability (cor.)	ElGamal + CP	ElGamal correctness; Theorem 6	DLog, ROM
Theorem 7 (mint addressing + anti-framing)	ElGamal + Okamoto	group axioms; Schnorr/Okamoto soundness	yes (binding)
Theorem 8 (deposit-coupling soundness)	Okamoto-in-SNARK + membership	SNARK knowledge soundness, DLog, ROM	yes
Theorem 9 (depositor-binding soundness)	Okamoto + membership + Schnorr	SNARK knowledge soundness, Schnorr EUF-CMA (DLog + ROM)	yes
Theorem 10 (nullifier + graph privacy)	Poseidon-PRF	ROM (Poseidon pre-image)	ROM
Theorem 11 (mutual decryptability)	ElGamal	ElGamal correctness; Schnorr EUF-CMA	DLog, ROM

Every assumption on the "yes" rows is already load-bearing for Ethereum itself (ECDSA on secp256k1 assumes DLog; every pairing-based SNARK verifier on L1 uses the same ROM reductions; every existing BLS-based rollup assumes q-SDH-class primitives). Alberta Buck introduces *no novel* cryptographic assumption.

7 Implementation Notes

7.1 What the Contract Must Enforce

The theorems assume a well-formed verifier. Three conditions on the on-chain code are load-bearing:

1. **Registration:** $\sigma'_1 \neq O$. Required by Theorem 1's non-degeneracy argument. A missing check admits the trivial signature attack (see Identity doc, Attack 9 / Link 1).
2. **Registration:** G_2 **subgroup check**. The `ecPairing` precompile (post-EIP-197) validates that X, Y lie in the correct prime-order subgroup of G_2 . Without this, small-subgroup attacks on the twist curve could forge pairings. Theorem 1 is stated over the correct subgroup; enforcement is the precompile's responsibility.
3. **Verification:** E_{alice} **read from storage, not calldata**. Theorem 6's reduction routes through the registration-time binding of E_{alice} to issuer-certified m . If Alice supplies E_{alice} via calldata the binding is severed and Theorem 6 does not apply.

7.2 What the Prover Must Enforce

- **Uniform randomness for t, k_1, k_2, r_b .** Theorems 2 and 5 depend on uniform distributions. A biased RNG converts statistical unlinkability into a detectable correlation and converts perfect HVZK into approximate HVZK with an adversarial advantage proportional to the bias.
- **Fiat-Shamir domain separation.** The hash input must include all public inputs and context (`msg.sender`, `spender`, `chainid`). Omitting any one opens a cross-context replay path; the forking lemma reduction is unaffected, but the NIZK is no longer bound to the intended transaction.

7.3 Scope Not Covered Here

- *Knowledge soundness* of the registration-time NIZK that binds σ' to E_{alice} : this is a standard Schnorr-family sigma protocol over a linear pairing relation (see alberta-buck-identity.org, "One-Time (PS Signature + NIZK Verification at Registration)") and follows the same completeness / soundness / ZK template as Theorems 3-5. A separate write-up at the same level of detail is the natural next step.
- *Circuit soundness* of the BUCK Notes spend: Part IV proves that the deposit-coupling sigma (A1/A2), the depositor-binding sigma (B1), and the committed-point membership proof imply the desired guarantees *assuming* knowledge soundness of the underlying SNARK system. Soundness of the *proof system itself* (Groth16 / PLONK / STARK) depends on the chosen proof system's trusted setup (if any) and circuit correctness; this is orthogonal to the contents of this document.
- *Collision-resistance and hiding* of the Poseidon commitment hash H_{cm} and pseudorandomness of the Poseidon nullifier PRF. Poseidon is modelled as a random oracle in the SNARK circuit for the double-spend and commitment-binding arguments; its concrete security on BN254 has been independently studied and is assumed here. See the Poseidon-hash cryptanalysis literature for current best attacks and parameter choices.

7.4 The Long-Horizon Threat: Harvest-Now, Decrypt-Later

Every unlinkability guarantee in this document that rests on *ElGamal* – cross-account unlinkability, the *issuer-blindness* of the identity layer (a compelled or breached issuer cannot map its issued identities to chain accounts), and the third-party non-disclosure of A2 Notes – is *computational*, reducing to DDH (hence DLog) on $\mathbb{G}_1$. Only Theorem 2 (PS rerandomization unlinkability) is information-theoretic and survives unconditionally. This asymmetry has a sharp consequence against a powerful, patient adversary (a state or military actor with archival access).

The on-chain credentials $E_{\text{addr}} = (rG, M + rpk)$ are permanent public history. An adversary who learns the *plaintext* identity points $\{M_i\}$ – paradigmatically a **compelled or breached issuer**, whose KYC records *are* exactly that set – cannot link them to chain accounts *today*: testing whether a given E encrypts a known M_i is a plaintext-checking attack, i.e. deciding whether $(G, pk, R, C - M_i)$ is a DDH tuple, infeasible under DDH (this is the load-bearing observation behind the Identity doc’s privacy-boundary table and the paper’s *Issuer-blindness* theorem). But a future discrete-log break – in particular a cryptographically relevant quantum computer running Shor against $\mathbb{G}_1$ – collapses that DDH decision into a cheap test, retroactively de-anonymising *all past* identity and A2-Note traffic from the harvested $\{M_i\}$ plus the archived chain. The PS layer’s statistical unlinkability is unaffected, but PS unlinkability does not hide the $M \leftrightarrow E_{\text{addr}}$ relation; only the ElGamal layer does, and only computationally.

Two points are worth recording for the threat model:

1. **Epoch expiry does not bound it.** Epoch-based credential renewal (Identity doc) limits the value of breaking a *single* credential’s pairing equation, but the harvest target here is the persistent $M \leftrightarrow E_{\text{addr}}$ link, which a quantum DLog break recovers wholesale, expired or not.
2. **The migration is encryption-layer, not pairing-layer.** The BLS12-381 path (EIP-2537, "alt_bn128 Security Horizon" in the Identity doc) hardens *pairing* security for PS verification; it does nothing for the ElGamal confidentiality that DDH underwrites. Post-quantum unlinkability requires replacing the $\mathbb{G}_1$ ElGamal credential with a post-quantum encryption (a lattice-based KEM/commitment, with a matching re-encryption-equality argument to preserve the **approve** handshake) – a substantial identity-layer redesign, and the right place to invest if the threat model includes a quantum adversary with archival access to the issuer’s records.

This is a property of *every* DDH/DLog-based confidential-payment system, not of Alberta Buck specifically; it is recorded here because the issuer-breach corpus makes the harvest concrete and names the one party whose compromise hands an adversary the full plaintext set $\{M_i\}$.